

Cyberster Internship Program

Internship Report

TRACK: BLUE TEAM

NAME: MUHAMMAD RAZA

INSTRUCTOR: ABDULLAH ZIA

ROLL NUMBER: CSI-B1-121

WEEK 6: Insider Threat Simulation and Incident Response

PHASE 0 — PRE-ENGAGEMENT SETUP

Objective

The objective of this phase was to prepare a controlled and monitored lab environment capable of detecting file-based malicious activities in real time. This involved configuring a dedicated staging directory on the Windows endpoint and integrating it with the Wazuh File Integrity Monitoring (FIM) module. The purpose of this setup was to ensure that any unauthorized file creation, modification, or deletion performed during the simulated insider threat attack would be captured, logged, and forwarded to the Security Information and Event Management (SIEM) system for analysis.

This phase is critical because the effectiveness of the entire investigation depends on the visibility established before the attack. Without proper monitoring, malicious actions would leave little to no trace, making detection and forensic analysis impossible.

Environment Overview

The lab environment was composed of three primary systems, each performing a distinct role within the simulation:

- ### Windows Virtual Machine (Victim System):
  Served as the endpoint where the insider threat activity was executed. This system had the Wazuh agent installed and was responsible for generating telemetry data based on file system changes.

```
C:\Users\Mrym>ipconfig

Windows IP Configuration

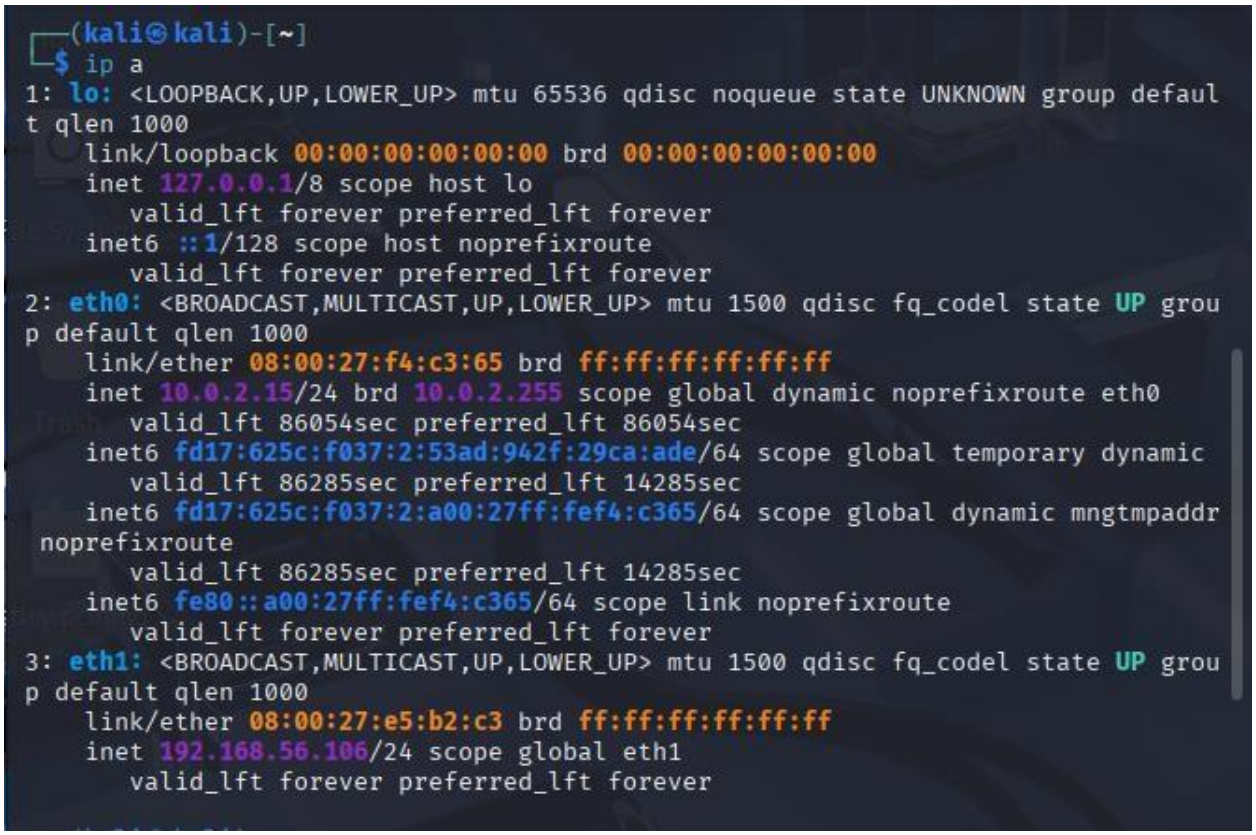

Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : fd17:625c:f037:2:79e5:e7b8:a855:1d77
   Temporary IPv6 Address. . . . . . : fd17:625c:f037:2:85fe:820b:e973:c1a4
   Link-local IPv6 Address . . . . . : fe80::7a25:cb8f:c14:78a2%9
   IPv4 Address. . . . . . . . . . . : 10.0.2.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::2%9
                                       10.0.2.2

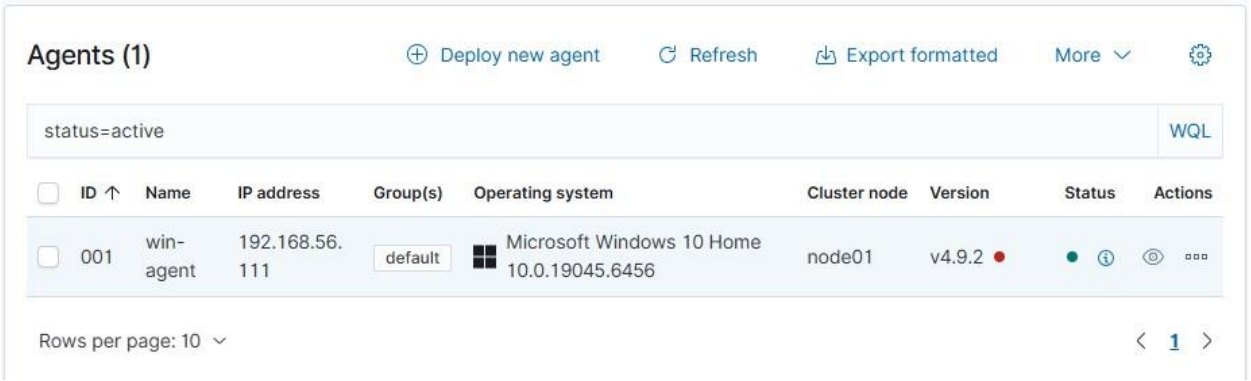
Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::2804:59e3:1270:3810%13
   IPv4 Address. . . . . . . . . . . : 192.168.56.111
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
```

- ## Kali Linux Virtual Machine (Attacker System):
  Functioned as the attacker-controlled machine used for receiving exfiltrated data. It also played a role in later forensic validation.



- ## Wazuh SIEM (Manager + Dashboard):
  Deployed using a pre-configured OVA, this system acted as the centralized logging and monitoring platform. It collected, processed, and visualized security events generated by the Windows endpoint.

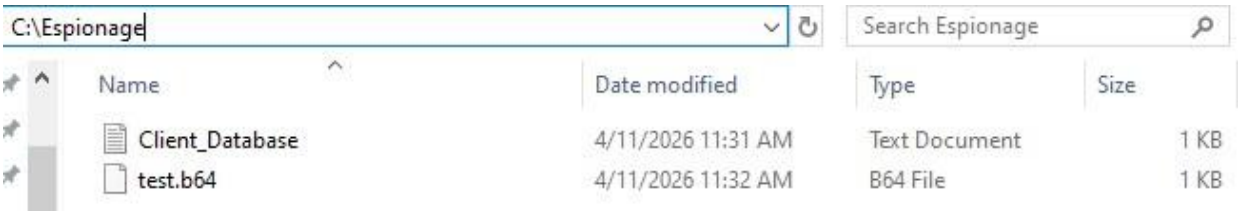


```
[root@wazuh-server ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:7a:6b:75 brd ff:ff:ff:ff:ff:ff
    altname enp0s8
    inet 192.168.56.110/24 metric 1024 brd 192.168.56.255 scope global dynamic eth0
       valid_lft 595sec preferred_lft 595sec
    inet6 fe80::a00:27ff:fe7a:6b75/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:da:a6:69 brd ff:ff:ff:ff:ff:ff
    altname enp0s17
    inet 10.0.2.15/24 metric 1024 brd 10.0.2.255 scope global dynamic eth1
       valid_lft 86395sec preferred_lft 86395sec
    inet6 fd17:625c:f037:2:a00:27ff:feda:a669/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86395sec preferred_lft 14395sec
    inet6 fe80::a00:27ff:feda:a669/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

This architecture simulates a real-world Security Operations Center (SOC) environment where endpoints continuously report activity to a centralized SIEM for detection and analysis.

Step 1: Creation of Staging Directory

A dedicated directory was created on the Windows system to simulate a sensitive data staging location:



This directory was intentionally chosen to represent a non-standard folder where an insider might temporarily store confidential data before exfiltration.

The directory was created using administrative privileges to ensure full access and control. This step established the target location for all subsequent attacker actions, including file creation, encoding, and deletion.

Step 2: Wazuh Agent Configuration

To enable monitoring of the staging directory, the Wazuh agent configuration file was modified. The configuration file is located at:

```
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mrym>"C:\Program Files (x86)\ossec-agent\ossec.conf
```

Within this file, the <syscheck> module is responsible for File Integrity Monitoring. The following configuration entry was added:

```
<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>43200</frequency>
<directories check_all="yes" realtime="yes">C:\Espionage</directories>
```

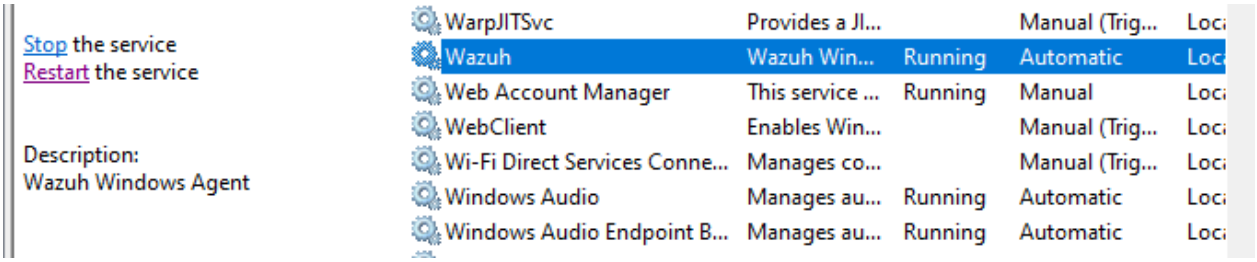
Explanation of Configuration Parameters

- **check_all="yes"**
  This parameter ensures that all file attributes are monitored, including hashes, permissions, ownership, and timestamps. This provides comprehensive visibility into any changes occurring within the directory.
- **realtime="yes"**
  Enables real-time monitoring using filesystem hooks. Without this setting, Wazuh would rely on periodic scans (default ~12 hours), which would delay detection significantly.
- **Directory Path (C:\Espionage)**
  Specifies the exact folder to be monitored. Any mismatch in the path (including case sensitivity or syntax errors) would result in the directory being ignored silently.

This configuration ensures that every file-related event within the staging directory is immediately captured and forwarded to the SIEM.

Step 3: Agent Service Restart

After modifying the configuration file, the Wazuh agent service was restarted using the Windows Services Manager (services.msc).



This step is mandatory because the Wazuh agent only loads its configuration during startup. If the service is not restarted, the new monitoring rules will not take effect.

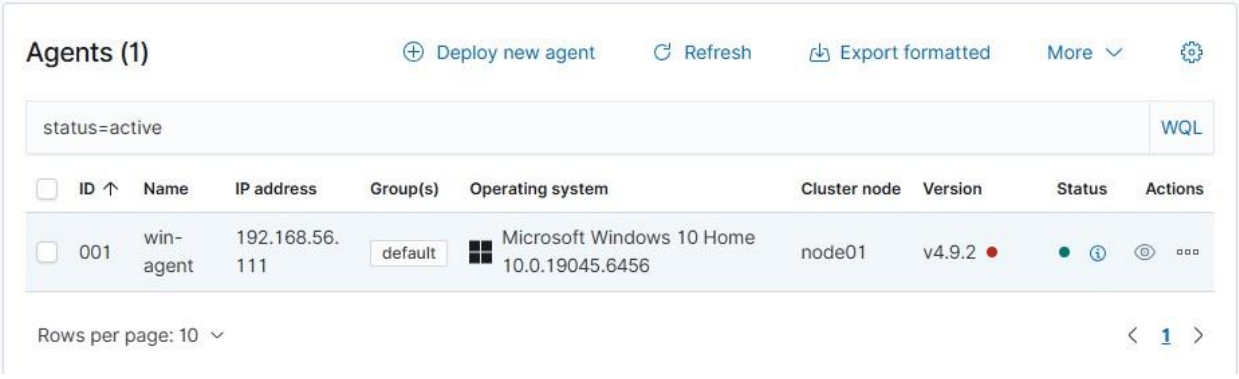
Failure to perform this step is a common issue that results in missing FIM alerts during attack execution.

Step 4: Verification of Agent Connectivity

Following the configuration and restart, the Wazuh dashboard was accessed to verify that the Windows agent was actively communicating with the Wazuh manager.

The verification process included:

- Confirming the agent status as **Active**
- Ensuring recent heartbeat logs were being received
- Validating that the system appeared in the agent inventory list



This step confirmed that the endpoint was successfully integrated into the SIEM environment and ready to generate security events.

Result

The pre-engagement setup was successfully completed. The staging directory was created, and real-time File Integrity Monitoring was configured and validated. The Wazuh agent was actively reporting to the SIEM, ensuring full visibility into file system activity within the monitored directory.

This configuration enabled the capture of all relevant events during the attack phase, including file creation, encoding, and deletion, forming the foundation for the subsequent detection and investigation processes.

PHASE 1 — RED TEAM: ATTACK EXECUTION

Objective

The objective of this phase was to simulate a realistic insider threat scenario from the perspective of a malicious user with legitimate access to a corporate system. The attacker's goal was to stage sensitive data, obscure it to avoid detection, exfiltrate it to an external system, and finally remove all traces of the activity.

This phase was designed to replicate real-world attack techniques where adversaries leverage trusted system tools rather than deploying external malware. Such techniques are commonly referred to as Living-off-the-Land (LotL) attacks, where built-in utilities are abused to bypass traditional security controls.

The execution of this phase provided the necessary artifacts and logs that would later be analyzed during the Blue Team investigation.

Attack Scenario Overview

The simulated attack followed a structured sequence of actions:

1. Establish a listener on the attacker system
2. Create a sensitive data file on the victim machine
3. Encode the data to obscure its contents
4. Exfiltrate the encoded data over HTTP
5. Delete all evidence to evade detection

Each step corresponds to a specific tactic in the MITRE ATT&CK framework**,** allowing the attack to be mapped to standardized threat behaviors.

Tools and Techniques Used

- **PowerShell (Windows):** Used for file creation, data manipulation, and exfiltration
- **certutil.exe:** A legitimate Windows utility abused for Base64 encoding
- **Netcat (Kali Linux):** Used as a listener to receive exfiltrated data
- **HTTP Protocol:** Used for covert data transmission
- **Wazuh FIM:** Passively monitored all file-related activities

Step 1 — Attacker Listener Setup (Kali Linux)

The attack began by preparing the attacker machine to receive exfiltrated data. A Netcat listener was started on a specific port:



Explanation

Netcat is a lightweight networking utility capable of reading and writing data across network connections. In this scenario, it was used to simulate an attacker-controlled server waiting to receive stolen data.

- **-l** → Listen mode
- **-v** → Verbose output
- **-n** → No DNS resolution (faster execution)
- **-p 8080** → Port number

Port **8080** was chosen because it is commonly used for HTTP traffic and is less likely to be blocked in internal networks compared to non-standard ports.

Security Insight

Using Netcat allows attackers to quickly establish a data exfiltration channel without deploying complex infrastructure. Because it does not require installation on most Linux systems, it is a preferred tool for quick and stealthy operations.

Step 2 — Data Staging on Windows Endpoint

The attacker created a file containing sensitive information within the monitored directory:

Explanation

The Set-Content cmdlet in PowerShell is used to create or overwrite files. In this case, it was used to simulate the creation of a confidential database file containing sensitive client information.

This step represents **data staging**, where attackers prepare information for extraction.

Step 3 — Data Obfuscation using certutil (Base64 Encoding)

The staged file was encoded using the Windows utility certutil.exe:

Explanation

certutil.exe is a legitimate Windows tool used for certificate management. However, it can also encode files into Base64 format.

Base64 encoding transforms binary or text data into an ASCII format, making it appear as unreadable text. This does not encrypt the data but makes it less obvious during casual inspection.

Why Attackers Use Base64

- Evades simple content inspection
- Makes data appear harmless (like system cache)
- Bypasses basic detection mechanisms

Security Insight

This step maps to MITRE ATT&CK T1027 — Obfuscated Files or Information.
Attackers frequently use encoding techniques to bypass detection systems that rely on keyword matching or plaintext inspection.

Step 4 — Data Exfiltration via HTTP POST

The encoded data was transmitted to the attacker machine using PowerShell:

Explanation

- Get-Content -Raw reads the entire file as a single string
- Invoke-WebRequest sends an HTTP request to a remote server
- The -Method Post option is used to send data in the request body

This simulates real-world data exfiltration over HTTP, which is one of the most common techniques used by attackers.

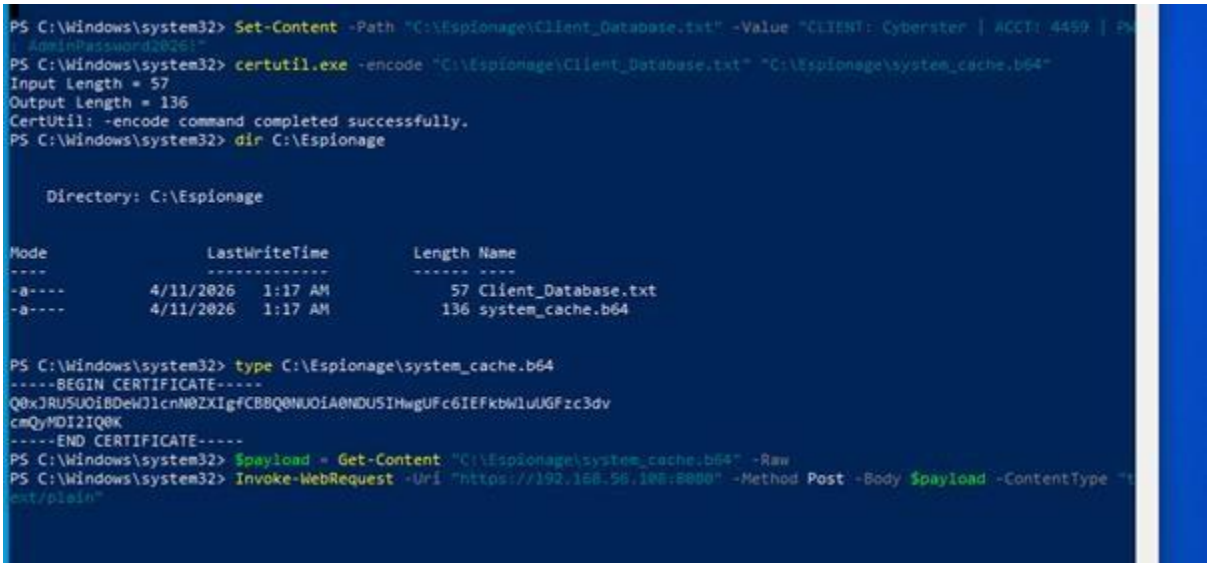
Step 5 — Anti-Forensics: Evidence Deletion

After successfully exfiltrating the data, the attacker removed all files:

Explanation

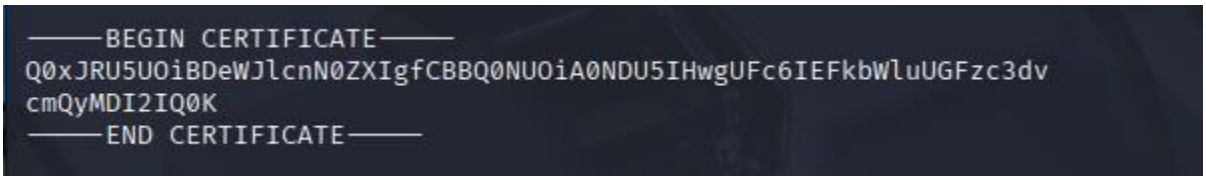
The Remove-Item cmdlet deletes files and directories. The -Force flag ensures that even hidden or protected files are removed without prompting.

This step attempts to eliminate any evidence of the attack from the system.



Step 6: Verify Data Reception on Kali

On the Kali VM, the netcat listener received the incoming HTTP POST request containing the Base64 encoded payload. The received data showed the full HTTP headers and the encoded certificate formatted Base64 content, confirming successful exfiltratio



Phase 2 — Blue Team: Detection and Investigation

Task 1: Wazuh Rule Trigger Analysis

1 — File Creation Rule Trigger

Event

File created:

t	full_log	File 'c:\espionage\client_database.txt' added Mode: realtime
t	id	1775889533.3883468
t	input.type	log
t	location	syscheck
t	manager.name	wazuh-server
t	rule.description	File added to the system.
#	rule.firedtimes	1
t	rule.gdpr	II_5.1.f
t	rule.gpg13	4.11
t	rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
t	rule.hipaa	164.312.c.1, 164.312.c.2
t	rule.id	554
#	rule.level	5
◔	rule.mail	false
t	rule.nist_800_53	SI.7
t	rule.pci_dss	11.5
t	rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t	syscheck.attrs_after	ARCHIVE
		added
t	syscheck.path	c:\espionage\client_database.txt
t	syscheck.sha1_after	399e806e3884512fa92be0bda5e71a843489ba20
t	syscheck.sha256_after	14fc142301b03ebefcfcb3a1e32c85df5bbbe99a139b5647a6bc0a700ce51864
#	syscheck.size_after	57
t	syscheck.uid_after	S-1-5-32-544
t	syscheck.uname_after	Administrators
t	syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, READ_CONTROL, SYNCHRONIZE, READ_DATA, READ_EA, EXECUTE, READ_ATTRIBUTES, DELETE, READ_CONTROL, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES
t	syscheck.win_perm_after.name	Administrators, SYSTEM, Users, Authenticated Users
🗓	timestamp	Apr 10, 2026 @ 23:38:53.248

Why This Rule Triggered

Wazuh File Integrity Monitoring (FIM) detected **a** new file creation inside a monitored directory.

This rule alone is low-to-medium severity, but becomes important when:

- File name is sensitive
- Directory is unusual
- Followed by encoding or deletion

2 — File Modification / Encoding Rule Trigger

Event

Modified

t	full_log	File 'c:\espionage\client_database.txt' modified Mode: realtime Changed attributes: mtime Old modification time was: '1775888625', now it is '1775889114'
t	id	1775889533.3884757
t	input.type	log
t	location	syscheck
t	manager.name	wazuh-server
t	rule.description	Integrity checksum changed.
#	rule.firedtimes	1
t	rule.gdpr	II_5.1.f
t	rule.gpg13	4.11
t	rule.groups	ossec, syscheck, syscheck_entry_modified, syscheck_file
t	rule.hipaa	164.312.c.1, 164.312.c.2
t	rule.id	550
#	rule.level	7
◕	rule.mail	false
t	rule.mitre.id	T1565.001
t	rule.mitre.tactic	Impact
t	rule.mitre.technique	Stored Data Manipulation
t	syscheck.path	c:\espionage\client_database.txt
t	syscheck.sha1_after	399e806e3884512fa92be0bda5e71a843489ba20
t	syscheck.sha256_after	14fc142301b03ebefcfcb3a1e32c85df5bbbe99a139b5647a6bc0a700ce51864
#	syscheck.size_after	57
t	syscheck.uid_after	S-1-5-32-544
t	syscheck.uname_after	Administrators
t	syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, READ_CONTROL, SYNCHRONIZE, READ_DATA, READ_EA, EXECUTE, READ_ATTRIBUTES, DELETE, READ_CONTROL, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES
t	syscheck.win_perm_after.name	Administrators, SYSTEM, Users, Authenticated Users
🗓	timestamp	Apr 10, 2026 @ 23:38:53.284

Why This Rule Triggered

- New .b64 file created
- File transformation occurred
- Directory contents

changed This is where real

detection starts. Indicators:

- Use of .b64 extension
- Presence of encoded data
- Possible obfuscation attempt

3 — File Deletion Rule Trigger (CRITICAL)

Event

Deletion

t	decoder.name	syscheck_deleted
t	full_log	File 'c:\espionage\client_database.txt' deleted Mode: realtime
t	id	1775888175.3741394
t	input.type	log
t	location	syscheck
t	manager.name	wazuh-server
t	rule.description	File deleted.
#	rule.firedtimes	4
t	rule.gdpr	II_5.1.f
t	rule.gpg13	4.11
t	rule.groups	ossec, syscheck, syscheck_entry_deleted, syscheck_file
t	rule.hipaa	164.312.c.1, 164.312.c.2
t	rule.id	553
#	rule.level	7
◕	rule.mail	false
t	rule.mitre.id	T1070.004, T1485
t	rule.mitre.tactic	Defense Evasion, Impact
t	rule.mitre.technique	File Deletion, Data Destruction
t	rule.nist_800_53	SI.7

t syscheck.path	c:\espionage\client_database.txt
t syscheck.sha1_after	399e806e3884512fa92be0bda5e71a843489ba20
t syscheck.sha256_after	14fc142301b03ebefcfcb3a1e32c85df5bbbe99a139b5647a6bc0a700ce51864
# syscheck.size_after	57
t syscheck.uid_after	S-1-5-32-544
t syscheck.uname_after	Administrators
t syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, READ_CONTROL, SYNCHRONIZE, READ_DATA, READ_EA, EXECUTE, READ_ATTRIBUTES, DELETE, READ_CONTROL, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES
t syscheck.win_perm_after.name	Administrators, SYSTEM, Users, Authenticated Users
timestamp	Apr 10, 2026 @ 23:16:15.092

Why This Rule Triggered

Wazuh detected mass deletion of files in monitored directory.

The deletion of files immediately after encoding activity suggests intentional anti-forensic behavior aimed at removing evidence.

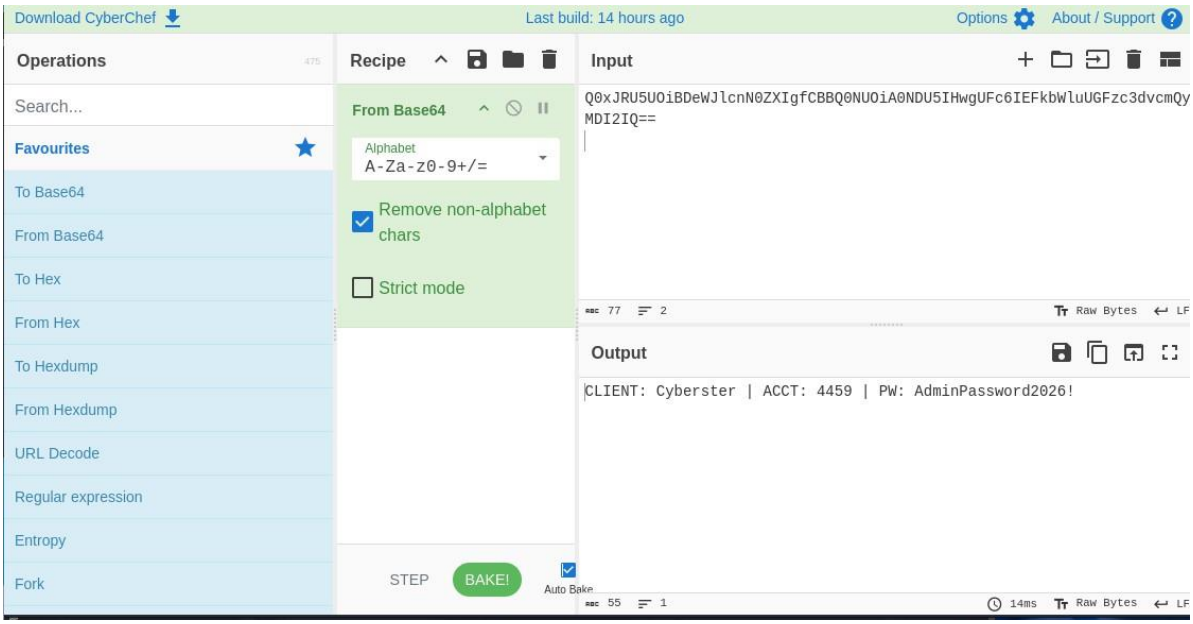
Rule	Meaning	Alone
554	File Created	Normal
550	File Modified	Normal
553	File Deleted	Normal

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Apr 10, 2026 @ 23:38:53.284	win-agent	c:\espionage\client_database.txt	modified	Integrity checksum changed.	7	550
Apr 10, 2026 @ 23:38:53.248	win-agent	c:\espionage\client_database.txt	added	File added to the system.	5	554
Apr 10, 2026 @ 23:16:15.092	win-agent	c:\espionage\client_database.txt	deleted	File deleted.	7	553

Task 2: Evidence Decoding with CyberChef

The Base64-encoded data received by the attacker's listener was decoded using CyberChef (https://gchq.github.io/CyberChef/). The Base64 content was pasted into the input field after removing the certificate header (-----BEGIN CERTIFICATE-----) and footer (---END CERTIFICATE---------) lines that certutil adds during encoding. The From Base64 recipe was applied, and the decoded output revealed the original plaintext content.

This confirmed that the exfiltrated data contained sensitive client credentials, validating the insider threat scenario and providing critical forensic evidence for the incident response report.



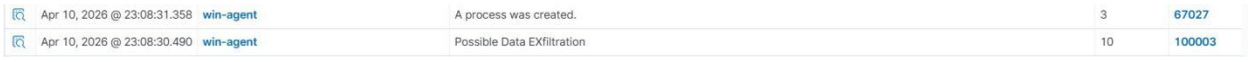
Task 3: Detection Engineering — Custom Wazuh Rule

The final investigative task was to engineer a custom Wazuh detection rule that would automatically alert on the specific attack pattern observed: the creation of Base64-encoded files within the Espionage directory. This rule would provide high-confidence detection of potential data exfiltration staging activity.

Initial Rule Development

The custom rule was developed in /var/ossec/etc/rules/local_rules.xml on the Wazuh Manager. The initial approach used a element to match the syscheck.path field against a regex pattern for
.b64 files in the Espionage directory, chained to Rule 554 (File added) via .

```
<rule id="100003" level="10">
<if_sid>554</if_sid>
<field name="file" type="pcre2">(?i)C:\\Espionage\\.+\.b64</field>
<description>Possible Data EXfiltration</description>
<mitre>
<id>T1027</id>
</mitre>
</rule>
```




```
Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> certutil.exe -encode "C:\Espionage\Client_Database.txt" "C:\Espionage\test.b64"
Input Length = 57
Output Length = 136
CertUtil: -encode command completed successfully.
PS C:\Windows\system32>
```

```
[root@wazuh-server ~]# sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -i "espionage"
File 'c:\espionage\test.b64' added
```

Phase 3 — Incident Response Report

Executive Summary

An authorized user with administrator privileges created a sensitive data file (Client_Database.txt) containing client credentials within a staging directory (C:\Espionage), encoded the file to Base64 format using the legitimate Windows utility certutil.exe, exfiltrated the encoded data to an external host (192.168.35.128:8080) via HTTP POST using PowerShell's Invoke-WebRequest, and subsequently deleted all evidence from the staging directory.

The incident was detected through Wazuh File Integrity Monitoring (FIM) alerts that captured file creation and deletion events in real-time. Evidence recovery confirmed the exfiltration of client credentials including account numbers and passwords. A custom detection rule (Rule ID 100500) was developed and deployed to provide automated alerting for this specific attack pattern in future incidents.

Detection and Analysis (MITRE ATT&CK Mapping)

The attack chain was mapped to the following MITRE ATT&CK techniques, providing a standardized framework for understanding the threat actor's behavior:

Technique	Tactic	Observed Behavior	Tool Used
T1074 — Data Staging	Collection	Creation of sensitive file in staging directory	PowerShell (Set-Content)
T1027 — Obfuscated Files or Information	Defense Evasion	Base64 encoding of data	certutil.exe
T1048 — Exfiltration Over Alternative Protocol	Exfiltration	Data transmitted via HTTP POST	PowerShell (Invoke-WebRequest)
T1070.004 — Indicator Removal (File Deletion)	Defense Evasion	Deletion of all staged files	PowerShell (Remove-Item)

Wazuh Rule Trigger Analysis

The detection of this incident relied on multiple Wazuh rules triggered during different stages of the attack.

Rule 554 — File Added

- Triggered when:
  - Client_Database.txt was created
  - system_cache.b64 was generated

Interpretation:

Indicates data staging and preparation activity. Individually low severity, but significant when correlated.

Rule 550 — File Modified

- Triggered during encoding process

Interpretation:

Represents transformation of data, consistent with obfuscation behavior.

Rule 553 — File Deleted

- Triggered when all files in C:\Espionage were removed

Interpretation:

Strong indicator of anti-forensic activity. High confidence of malicious intent when following encoding.

Custom Rule 100005— Suspicious Encoding Activity

- Triggered on .b64 file creation within monitored directory

Logic Summary:

- Detect Base64 file creation
- Identify suspicious naming patterns
- Increase alert severity

Interpretation:

Provides proactive detection of obfuscation attempts using legitimate tools.

Correlation Insight

While individual alerts appeared benign, their sequence revealed a complete attack chain:

File Creation → Encoding → Exfiltration → Deletion

This pattern confirms malicious activity consistent with insider data exfiltration.

Indicators of Compromise (IOCs)

Indicator Type	Value	Context
File Path	C:\Espionage\Client_Database.txt	Staging file
File Path	C:\Espionage\system_cache.b64	Encoded file
File Extension	.b64	Obfuscation indicator
Tool	certutil.exe	LOLBin abuse
Tool	PowerShell (Invoke-WebRequest)	Exfiltration
Network	HTTP POST (Port 8080)	Data transfer channel
Behavior	Bulk file deletion	Anti-forensics

Containment, Eradication, and Recovery

Containment

- Immediately disable the compromised user account
- Isolate the affected endpoint from the network
- Block outbound communication to suspicious IPs and ports
- Preserve all logs and forensic artifacts

Eradication

- Perform full system scan for persistence mechanisms
- Analyze PowerShell execution logs (if available)
- Investigate additional directories for hidden staging activity
- Remove unauthorized scripts or scheduled tasks

Recovery

- Reset all compromised credentials
- Restore system from a clean backup
- Validate system integrity post-recovery
- Reconnect system to network after verification

Post-Incident Recommendations and Lessons Learned

Detection Improvements

- Deploy custom Wazuh rule (ID 100500) in production
- Enable PowerShell Script Block Logging (Event ID 4104)
- Monitor certutil.exe usage via Sysmon
- Integrate network monitoring tools for HTTP inspection

Policy Improvements

- Enforce least privilege access control
- Restrict execution of LOLBins via AppLocker/WDAC
- Implement Data Loss Prevention (DLP) policies
- Conduct insider threat awareness training

Key Lessons Learned

1. **FIM provides visibility but not completeness**
   File monitoring alone cannot detect network-based exfiltration.
2. **Legitimate tools can be weaponized**
   Detection must focus on behavior, not just binaries.
3. **Correlation is critical**
   Individual alerts are weak; combined alerts reveal attacks.
4. **Anti-forensics is effective against local analysis**
   Centralized logging is essential for post-incident investigation.

Conclusion

This incident demonstrates a realistic insider threat scenario where an attacker successfully leveraged legitimate tools to perform data staging, obfuscation, exfiltration, and evidence removal. Although traditional detection mechanisms failed to identify the full attack in real time, Wazuh FIM provided sufficient visibility to reconstruct the attack through log correlation.

The development of a custom detection rule significantly enhances the organization's ability to detect similar threats in the future. However, the incident highlights the necessity of a layered security approach that includes endpoint monitoring, network analysis, and behavioral detection.

This exercise reflects real-world SOC operations, emphasizing the importance of detection engineering, incident response planning, and continuous security improvement.